Projet - Circuits et architecture
À rendre : 19/12/2025 et 12/01/2026 à 18h

1 Présentation

Ce projet a un double but :

Câbler dans `logisim` un micro-processeur qui implante un sous-ensemble des instructions du micro-processeur LC-3, et

Programmer avec ces instructions un petit jeu de routines.

Le point de départ est le circuit `logisim` `LC-3-v0.circ`, que vous trouverez sur la page web du cours [1]. Il est **essentiel de modifier précisément ce fichier** pour réaliser votre projet.

Le projet comporte deux niveaux : un *minimum requis* et un *projet avancé*.

1.1 Instructions LC-3 à implémenter

Le circuit principal `main` utilise plusieurs sous-circuits, dont les plus importants sont : `ALU`, `BiClock`, `DecodeIR`, `GetAddr`, `NZP`, `RamCtrl`, `RegPC`, `WriteVal`. Les sous-circuits coloriés en noir dans `main` sont complètement câblés et **ne doivent pas être modifiés**. Le circuit `main` exécute correctement les instructions `NOT`, `ADD` et `AND` du LC-3.

Avant de commencer le câblage, vous devez comprendre le fonctionnement du circuit distribué. Une fois ce fonctionnement compris, vous devez implémenter un certain nombre d'instructions.

Minimum requis : vous devez modifier le câblage (et pas l'interface) des circuits coloriés en rouge afin d'implémenter les instructions `LEA`, `LDR`, `LD`, `ST`, `STR`, `BR` et `JMP`, ainsi que `XOR` et `POPCNT` décrites ci-dessous.

Projet avancé : vous devez compléter les modules `RegPC` et `WriteVal` afin d'implémenter les instructions `JSR` et `JSRR`. Vous pouvez aussi modifier *minimalement* le reste du circuit.

La sémantique de ces instructions est celle vue en cours [2], en particulier "Cours n° 6 : description du LC-3". Vous pouvez consulter également le site internet du livre de Patt & Patel [3], en particulier les annexes A et C. Les instructions `XOR` et `POPCNT` sont décrites dans les sous-sections suivantes.

1.1.1 Instruction XOR

On étend le LC-3 avec une nouvelle instruction XOR, permettant de calculer le ou exclusif bit à bit de deux registres 16 bits, et on implémente cette instruction en `logisim`. Pour cela, il faut modifier le module `ALU` uniquement. L'usage des portes `XOR` de `logisim` est autorisé.

Codage de l'instruction : l'opcode de `XOR` sera celui de `NOT`. Il faut bien sûr que tous les programmes utilisant le NOT continuent à fonctionner avec le LC-3. Vous devrez donc expliquer dans le rapport pourquoi cela est possible. L'instruction `XOR` doit modifier `nzp`. Elle supportera deux modes d'adressage calqués sur ceux de `AND` :

15			12	11	9	8	6	5	4	3	2	0
1	0	0	1	DR		SR1		0	0	0	SR2	
1	0	0	1	DR		SR1		1	Imm5			

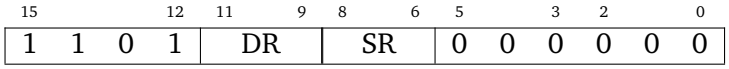
La première ligne correspond à l'adressage registre : `DR` reçoit le résultat du `XOR` bit à bit entre `SR1` et `SR2`. La deuxième ligne correspond à l'adressage immédiat : `DR` reçoit le résultat du `XOR` bit à bit entre `SR1` et `Imm5`.

1.1.2 Instruction pour le poids de Hamming

On étend également l'ensemble d'instructions du LC-3 avec une nouvelle instruction arithmético-logique `POPCNT` (pour *population count*), qui calcule le poids de Hamming d'un entier 16 bits.

Le poids de Hamming $H(n)$ d'un entier n est le nombre de bits à 1 dans la représentation binaire de n. Par exemple, $H(2) = H(4) = 1$ et $H(9) = H((1001)_2) = 2$ et $H(15) = H((1111)_2) = 4$. Cette opération est difficile à implémenter efficacement en logiciel, alors qu'elle est utile dans de nombreux domaines (codes, cryptographie, certaines structures de données, etc.). Il est donc utile de l'implémenter directement en matériel.

Le code opération (*opcode*) de cette instruction est celui disponible en LC-3, `1101`.



`POPCNT DR, SR`

Cette instruction agit uniquement sur les registres : le registre `DR` se voit affecté le poids de Hamming (nombre de bits à 1) de la valeur contenue dans le registre `SR`. Elle met également à jour les indicateurs `n`, `z` et `p`.

Là encore, il faut modifier le module `ALU` uniquement, même si l'usage de sous-modules est évidemment possible (et encouragé). L'usage de la porte `Bit Adder` de `logisim` est interdit, par contre celui de la porte `Adder` de `logisim` est autorisé. Vous devez **favoriser une implémentation à bas coût** de l'instruction, en termes de nombre de portes et de profondeur du circuit obtenu.

L'élégance et la simplicité de votre solution sera appréciée.

1.2 Programmation en LC-3 étendu

Le circuit construit vous servira pour tester un ensemble de routines écrites en assembleur LC-3 que vous devez implémenter, comme indiqué ci-dessous.

1.2.1 Manipulation de chaînes

Les chaînes de caractères seront codées comme des séquences de caractères terminées par le caractère nul. De plus, on va supposer que les caractères sont codées sur 16 bits[1]. Les programmes à coder ont un comportement similaire aux fonctions homonymes de la bibliothèque du langage C.

- `index` calcule en `R0` l'adresse de la première apparition d'un caractère dont le code est donné en `R2` pour une chaîne de caractères dont l'adresse de début est donnée en `R1` ; la valeur de `R0` est `0` si le caractère est absent.
- `strcpy` copie la chaîne source (y compris le caractère nul) dont l'adresse de début est donnée en `R1` dans la chaîne destination dont l'adresse est en `R2`.
- `strncpy` copie la chaîne source dont l'adresse de début est donnée en `R1` dans la chaîne destination dont l'adresse est en `R2` sur un nombre de caractères donné par le registre `R0`.

1. Cette convention est spécifique à LC-3, les caractères ASCII étant normalement représentés sur 1 octet.

1.2.2 Renversement des bits

La sous-routine `reverse` prend un argument entier $x = (b_{15} \dots b_0)_2$ sur 16 bits et renvoie en résultat le nombre $(b_0 \dots b_{15})_2$ ayant les mêmes bits que x mais dans l'ordre inverse : les bits de poids faible deviennent de poids fort et vice-versa. Par exemple :

$$\texttt{reverse}(24) = \texttt{reverse}((0000\,0000\,0001\,1000)_2) = (0001\,1000\,0000\,0000)_2 = 2^{12} + 2^{11} = 6144.$$

Intérêt : un décalage droit (division par 2) devient par renversement un décalage gauche, autrement dit un simple doublement.

1.2.3 *Bit scan*

Les sous-routines `clz` et `ctz` (pour *count leading zeros* et *count trailing zeros*) prennent un argument entier $x \neq 0$[2] et renvoient le nombre de zéros consécutifs de chaque côté de l'écriture binaire de x sur 16 bits : côté poids fort pour `clz` et côté poids faible pour `ctz`. Par exemple :

$$\begin{aligned}
\texttt{clz}(13) &= \texttt{clz}((0000\,0000\,0000\,1101)_2) = 12 \\
\texttt{ctz}(13) &= \texttt{ctz}((0000\,0000\,0000\,1101)_2) = 0 \\
\texttt{clz}(-8) &= \texttt{clz}((1111\,1111\,1111\,1000)_2) = 0 \\
\texttt{ctz}(-8) &= \texttt{ctz}((1111\,1111\,1111\,1000)_2) = 3.
\end{aligned}$$

Attention : vous devez implémenter `ctz` efficacement via les instructions `POPCNT` et `XOR`, en remarquant que $\texttt{ctz}(x) = \texttt{POPCNT}(x \oplus (x-1)) - 1$. En revanche, il n'existe pas de formule similaire pour `clz`.

1.2.4 Produit saturé

On veut programmer une fonction qui calcule le produit saturé et non signé de deux opérandes 16 bits. Si ces opérandes ont pour valeurs non signées a et b, on veut donc $\min(a \times b, 2^{16} - 1)$. Bien entendu, n'hésitez pas à utiliser l'instruction `POPCNT`, ainsi que vos sous-routines précédentes sur les bits.

Prédiction de débordement Écrivez un programme qui prédit si la multiplication des entiers non signés stockés dans R1 et R2 risque de déborder, en utilisant seulement l'information de la position de leur bit de poids fort (obtenue via `clz`). Selon les cas, on pourra soit déterminer avec certitude si le produit déborde ou non soit rester dans l'incertitude. Le résultat doit être stocké dans R0, avec l'encodage suivant :

- n'importe quelle valeur strictement négative : le produit ne déborde pas ;
- 0 : il y a un risque de débordement ;
- n'importe quelle valeur strictement positive : le produit déborde à coup sûr.

(Avancé) Produit saturé 16 bits Écrivez un programme qui calcule la multiplication saturée des entiers non signés stockés dans R1 et R2. Le résultat sera stocké dans R0. Vous êtes invités à implémenter un algorithme de *shift-and-add*[3].

2. Les *bit scans* de la valeur 0 ne sont pas définis. En pratique, les sous-routines doivent retourner une valeur arbitraire.
3. https://en.wikipedia.org/wiki/Multiplication_algorithm#Usage_in_computers

(Très avancé) Produit saturé 32 et 64 bits Écrivez maintenant deux programmes qui calculent respectivement la multiplication saturée 32 et 64 bits. Pour le premier, vous aurez besoin de 6 registres ($2 * 2$ pour les opérandes et 2 pour le résultat). Pour le second, les opérandes et le résultat seront stockés en mémoire. Notez que vous pouvez avantageusement exploiter le multiplieur 16 bits pour améliorer le prédicteur de débordement en le rendant déterministe !

1.3 Consignes

Pour la mise au point de vos programmes, nous vous recommandons d'utiliser les outils de simulation LC-3 `lc3tools` [4]. Nous attendons de vous :

(minimum requis) d'avoir codé ces fonctions comme autant de programmes indépendants,

(projet avancé) d'avoir codé ces fonctions comme des sous-routines qui ne modifient pas d'autres registres que ceux du résultat. Vous pouvez soit utiliser une mémoire de sauvegarde par sous-routine ou implémenter une pile.

2 Modalités pratiques

Le projet s'effectue *en binôme*. *Une pénalité de 30% sera appliquée à la note finale pour les groupes de 3 ou 1 étudiant(s). Les groupes de plus de 3 étudiants ne sont pas acceptés.*

2.1 Paquetage

Le projet à rendre doit contenir :

Circuit : Le fichier `logisim` `LC-3-v1.circ` (respectivement `LC-3-v2.circ`) contenant le circuit complété pour le rendu intermédiaire (respectivement rendu final) du projet. Vous devez écrire des programmes de test (de type `.mem`) pour chaque instruction.

N'hésitez pas à ajouter des commentaires explicatifs dans le circuit.

Programmes : Les fichiers assembleur LC-3 (de type `.asm`) avec le code des programmes demandés. Dans le code, indiquez les données de test que vous avez utilisées.

Rapport : Vous devez rendre un rapport expliquant la démarche suivie et l'avancement atteint. Il faut énumérer rapidement les changements du circuit original et montrer qu'ils réalisent bien les fonctions souhaitées. Votre rapport doit aussi comporter le code des programmes de test utilisés et justifier qu'ils illustrent le fonctionnement du circuit. Le rapport ne doit pas dépasser 5 pages grand maximum.

2.2 Remise du projet

Le rendu du projet se fera en deux étapes, par courriel[4] aux enseignants des travaux dirigés :

Rendu intermédiaire, le 19 décembre 2025 avant 18h : le circuit (fichier `LC-3-v1.circ`) avec les instructions `POPCNT` et `XOR` câblées et les fichiers `.asm` contenant le code LC-3 de `index`, `strcpy`, `strncpy`. **Les binômes seront figés** par ce rendu.

Rendu final, le 12 janvier 2026 avant 18h : le paquet complet avec les circuits LC-3 (fichiers `LC-3-v1.circ` et `LC-3-v2.circ`), et tous les programmes assembleur LC-3 (fonctions codées, les tests de ces fonctions et des circuits) et le rapport.

4. Aux adresses letouzey@irif.fr (étudiants de M1) et pierre-yves.piriou@edf.fr (étudiants EIDD).

2.3 Soutenance

Le projet donnera lieu à une présentation du compte-rendu de votre travail en soutenance (par binôme) en janvier. Vous disposerez de 10 minutes de présentation, laquelle sera suivie de 5 minutes d'interrogation. La qualité de la présentation, ainsi que votre aisance pour répondre aux questions seront apprécié. Mettez en valeur vos réalisations, présentez ce que vous avez mis en œuvre pour vous assurer que vous aviez atteint les objectifs, ce que vous avez trouvé particulièrement instructif et ce que vous auriez aimé approfondir. Rappelez vous que, bien que la note tiendra compte de la lisibilité du circuit, de la simplicité de la solution choisie et de la qualité rédactionnelle du rapport, c'est la présentation en soutenance de votre travail qui donnera à vos évaluateurs la première impression sur sa qualité d'ensemble.

Versions

20-11-2025 Version initiale.

Références

[1] https://moule.informatique.univ-paris-diderot.fr/guatto/archi-m1-2025

[2] Oliver Carton, Notes de cours, http://www.irif.fr/~carton/Enseignement/Architecture

[3] Yale N. Patt, Sanjay J. Patel, Introduction to Computing Systems : From Bits and Gates to C and Beyond, McGraw-Hill, http://highered.mcgraw-hill.com/sites/0072467509/

[4] https://github.com/chiragsakhuja/lc3tools